ДИСЦИПЛИНА	Технологии разработки приложений на базе фреймворков
ИНСТИТУТ	Институт перспективных технологий и индустриального программирования
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Практическая работа
ПРЕПОДАВАТЕЛЬ	Ящун Татьяна Викторовна
СЕМЕСТР	5 семестр, 2026-2027

ПРАКТИЧЕСКАЯ РАБОТА № 2
ОСНОВЫ PYTHON: КОЛЛЕКЦИИ, ФУНКЦИИ, ЦИКЛЫ И РАСШИРЕННЫЕ ВОЗМОЖНОСТИ PYTHON

ЦЕЛЬ РАБОТЫ

Освоить коллекции данных, функции, циклы и расширенные возможности языка Python, изучить работу с модулями и пакетами, исключениями и файлами, а также закрепить указанные знания при развитии индивидуального сквозного проекта, начатого в рамках Практической работы № 1.

СОДЕРЖАНИЕ

1. ЗАДАЧИ РАБОТЫ 2
2. ПОРЯДОК ВЫПОЛНЕНИЯ И ОФОРМЛЕНИЯ РАБОТЫ 2
3. МАТЕРИАЛЫ ЯНДЕКС ПРАКТИКУМА 5
4. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ 5
   4.1. Коллекции данных 5
   4.2. Итерации и циклы 6
   4.3. Функции 6
   4.4. Модули и пакеты 7
   4.5. Исключения 7
   4.6. Работа с файлами и контекстные менеджеры 7
   4.7. Расширенные возможности Python 8
5. ПРАКТИЧЕСКАЯ ЧАСТЬ. ВЫПОЛНЕНИЕ МАТЕРИАЛОВ ЯНДЕКС ПРАКТИКУМА 8
   5.1. Повторение и закрепление основных конструкций Python 9
   5.2. Проект «Холодильник» 9
   5.3. Расширение возможностей программы 10
   5.4. Тестирование и отладка программы 10
   5.5. Обновление файла README.md 11
   5.6. Фиксация результата в Git 11
6. ИНДИВИДУАЛЬНЫЙ СКВОЗНОЙ ПРОЕКТ 11
   6.1. Обязательная доработка индивидуального проекта 11

6.2. Пример доработки: проект «Система бронирования помещений» 13
6.3. Ожидаемый результат 26
7. РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ ПРАКТИЧЕСКОЙ РАБОТЫ 26
8. ФОРМА КОНТРОЛЯ И КРИТЕРИИ ОЦЕНКИ 27
8.1. Чек-лист выполнения материалов Яндекс Практикума 28
8.2. Чек-лист выполнения индивидуального проекта 28
8.3. Критерии оценки 29
8.4. Требования к отчету 29
9. КОНТРОЛЬНЫЕ ВОПРОСЫ 30
10. ИТОГ 32

1. ЗАДАЧИ РАБОТЫ

В ходе выполнения практической работы необходимо:

- завершить изучение основных конструкций языка Python, связанных с коллекциями, функциями и циклами;
- познакомиться с организацией Python-кода в виде модулей и пакетов;
- изучить обработку исключительных ситуаций;
- освоить работу с файлами и контекстными менеджерами;
- познакомиться с интроспекцией и другими расширенными возможностями Python;
- выполнить проектное задание Яндекс Практикума «Холодильник»;
- продолжить развитие индивидуального сквозного проекта, начатого в ПР1, с применением новых конструкций языка;
- привести код индивидуального проекта в соответствие с требованиями к качеству программного кода: оформление, проверка линтером flake8, аннотации типов;
- выполнить первичное тестирование проекта с использованием pytest;
- подготовить отчет о выполнении работы и представить результаты преподавателю.

2. ПОРЯДОК ВЫПОЛНЕНИЯ И ОФОРМЛЕНИЯ РАБОТЫ

1. Войдите в личный аккаунт Яндекс с использованием персональной Яндекс-почты.
2. Подключитесь к курсу «Бэкенд-разработка» на образовательной платформе Яндекс Практикума по предоставленной преподавателем ссылке.
3. Ознакомьтесь с содержанием практической работы. Перед началом работы ознакомьтесь:

- с целью и задачами практической работы;
- с перечнем материалов Яндекс Практикума, которые необходимо выполнить;
- с материалом сквозного примера индивидуального проекта;
- с требованиями к отчету;
- с формой контроля и перечнем контрольных вопросов.

4. Выполните материалы Яндекс Практикума. Завершите темы раздела 01, отмеченные на ПР1 как переносимые на ПР2, и выполните темы раздела 03. Состав выполняемых материалов по каждому разделу приведен ниже.

Раздел 01. «Основы Python» (продолжение). На ПР1 выполнены первые три темы раздела; на ПР2 изучение раздела завершается, после чего выполняется проектная работа «Холодильник»:

№	Тема	Выполнить на
1	Простые типы данных	ПР1
2	Ветвления	ПР1
3	Импортируемые типы данных	ПР1
4	Коллекции	ПР2
5	Итерации и циклы	ПР2
6	Проектная работа «Холодильник»	ПР2

Особое внимание в темах раздела уделяется:

- спискам, кортежам, множествам и словарям;
- циклам `for` и `while`;
- обработке и преобразованию данных с использованием функций и циклов;
- применению коллекций и циклов при выполнении проектной работы «Холодильник».

Раздел 02. «Настройка рабочего окружения». Материал раздела выполнен на ПР1; повторное прохождение тем не требуется — используются настроенные ранее:

- рабочее окружение;
- система контроля и управления версиями;
- требования к оформлению кода;
- средства отладки.

Примечание. Если работа выполняется на новом компьютере или окружение было повреждено, вернитесь к материалам раздела 02 и повторите настройку по методическим указаниям к ПР1, после чего продолжите выполнение ПР2.

Раздел 03. «Python: новый уровень» и «Расширенные возможности Python». Выполните все уроки перечисленных тем:

№	Тема	Урок (материал)	Выполнить на ПР2
1	Python: новый уровень	«Новая задача»	✓
2	Python: новый уровень	«Пользовательские модули и пакеты»	✓
3	Python: новый уровень	«Интроспекция»	✓
4	Python: новый уровень	«Документирование кода»	✓
5	Python: новый уровень	«Исключения»	✓
6	Python: новый уровень	«Доработка проекта»	✓

7	Python: новый уровень	«Работа с файлами»	✓
8	Python: новый уровень	«Контекстные менеджеры»	✓
9	Python: новый уровень	«Новый интерфейс для проекта»	✓
10	Python: новый уровень	«Практика по теме»	✓
11	Расширенные возможности Python	«Пространство имён и области видимости»	✓
12	Расширенные возможности Python	«Итераторы и генераторы»	✓
13	Расширенные возможности Python	«Lambda-функции»	✓
14	Расширенные возможности Python	«Декораторы»	ПР3

Важно. Материал о декораторах в ПР2 не выполняется. Он рассматривается в следующей практической работе в составе изучения объектно-ориентированного программирования и связанных с ним возможностей Python.

Отметьте выполненные материалы в чек-листе практической работы (подраздел 8.1).

5. Доработайте индивидуальный проект. Используя сквозной пример из раздела 6, переработайте начальный сценарий ПР1: переведите данные проекта в структуры-коллекции и реализуйте функции работы с помещениями и бронированиями (добавление, поиск, отбор и сортировка помещений, вывод списков, проверка вместимости и доступности, создание и отмена бронирований). Данные проекта сохраняйте в файлах формата JSON. Классы и объекты на данном этапе не используются.
6. Выполните тестирование и отладку программы (подраздел 5.4). Проверьте работу приложения на подготовленных данных, используйте отладчик IDE для пошагового выполнения и контроля значений переменных.
7. Приведите код проекта в соответствие с требованиями к качеству (подраздел 6.1): проверьте оформление кода, выполните проверку линтером flake8 и добавьте аннотации типов к основным функциям.
8. Выполните первичное тестирование проекта (подраздел 6.2): установите pytest, создайте каталог tests и реализуйте не менее 3–5 автоматизированных тестов для основных функций.
9. Обновите файл README.md проекта (подраздел 5.5): дополните описание реализованных функций, структуры проекта и формата данных.
10. Зафиксируйте результат в Git. После завершения разработки:

- проверьте состояние репозитория;
- добавьте изменения;
- создайте commit;

- проверьте историю изменений;
- при использовании удаленного репозитория отправьте изменения.

В репозитории должна быть сохранена версия индивидуального проекта, соответствующая ПР2.

11. Подготовьте отчет о выполнении работы в соответствии с требованиями подраздела 8.4 и представьте результаты преподавателю.

3. МАТЕРИАЛЫ ЯНДЕКС ПРАКТИКУМА

Практическая работа выполняется с использованием курса «Бэкенд-разработчик» Яндекс Практикума и продолжает работу с материалами, начатую в ПР1. Материалы курса выступают основным учебным ресурсом: теория, примеры и практические задания каждого раздела выполняются последовательно.

В рамках ПР2 используются разделы курса:

- Раздел 01. Основы Python (завершение тем, перенесенных с ПР1);
- Раздел 02. Настройка рабочего окружения (выполнен на ПР1);
- Раздел 03. «Python: новый уровень».

Примечание. Материалы Яндекс Практикума используются как основной учебный ресурс. Дополнительные материалы настоящих методических указаний применяются для закрепления: теоретические сведения приведены в разделе 4, развитие индивидуального проекта – в разделе 6.

4. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Перед выполнением практической части рекомендуется ознакомиться с краткими теоретическими сведениями. Они помогают восстановить контекст: показать, какие возможности Python осваиваются на ПР2, зачем они нужны и как связаны с проектом «Холодильник» и индивидуальным проектом. Приведенные сведения не заменяют материалы Яндекс Практикума, а организуют их изучение.

4.1. Коллекции данных

Коллекции — это структуры данных, предназначенные для хранения наборов значений. До настоящего момента программа индивидуального проекта работала с отдельными переменными; коллекции позволяют перейти к обработке наборов данных: перечню помещений, списку бронирований, словарю товаров. В Python используются четыре основные коллекции:

- список (`list`) — упорядоченная изменяемая коллекция; используется, когда важен порядок элементов и их состав может меняться;
- кортеж (`tuple`) — упорядоченная неизменяемая коллекция; подходит для фиксированных наборов значений;

- множество (`set`) — неупорядоченная коллекция уникальных элементов; применяется для проверки принадлежности и устранения повторов;
- словарь (`dict`) — коллекция пар «ключ — значение»; используется, когда элементы нужно находить по смысловому идентификатору.

Для выбора коллекции достаточно ответить на два вопроса: важен ли порядок элементов и как элементы будут искаться. Так, содержимое холодильника удобно представить словарем, ключом которого является название продукта, а значением — количество и срок годности; перечень бронирований — списком словарей.

4.2. Итерации и циклы

Циклы обеспечивают многократное выполнение действий и применяются совместно с коллекциями: перебор элементов, накопление результата, поиск и фильтрация данных. Совместное использование коллекций и циклов — основной прием обработки данных в программах на Python.

- цикл `for` перебирает элементы коллекции; количество повторений определяется самой коллекцией;
- цикл `while` повторяет действия, пока выполняется условие; применяется, когда количество повторений заранее неизвестно;
- функция `range()` используется для перебора целых чисел и формирования последовательностей;
- с помощью циклов обрабатываются словари: ключи, значения и пары «ключ — значение».

В проекте «Холодильник» циклы применяются при переборе продуктов и формировании отчета о содержимом, а в индивидуальном проекте — при обработке списков помещений и бронирований и при организации основного цикла меню.

4.3. Функции

Функция — это именованный блок кода, выполняющий конкретную операцию. Выделение функций уменьшает дублирование, упрощает отладку и позволяет собирать программу из логически завершенных частей. На ПР1 в начальном сценарии была реализована одна функция; на ПР2 функциональность программы сознательно распределяется между несколькими функциями.

- параметры задают исходные данные функции;
- инструкция `return` возвращает результат; функция без `return` возвращает `None`;
- одна функция должна выполнять одну логически завершенную операцию;
- документирующая строка (`docstring`) кратко описывает назначение функции, её параметры и результат.

При проектировании функций индивидуального проекта следует опираться на состав операций предметной области: проверка возможности бронирования, создание

брони, поиск и фильтрация данных — каждый из этих сценариев становится отдельной функцией.

4.4. Модули и пакеты

Модуль — это файл с расширением .py, содержащий определения функций и другие объекты языка. Пакет объединяет несколько модулей в общую структуру каталогов. Организация кода в виде модулей делает программу обозримой: каждый файл отвечает за свою часть функциональности, а подключение выполняется инструкцией import.

- стандартные модули Python подключаются инструкцией `import` и расширяют возможности программы;
- собственный модуль создается как обычный `.py`-файл и подключается по имени;
- функция `main()` служит точкой входа и организует взаимодействие с пользователем;
- разбиение на модули выполняется по смыслу: логика предметной области, работа с данными, вспомогательные функции.

Навык организации модулей напрямую используется в индивидуальном проекте: на ПР2 программа разделяется на несколько файлов, что готовит проект к дальнейшему усложнению.

4.5. Исключения

Исключение — это ошибка, возникающая во время выполнения программы: некорректный ввод пользователя, отсутствие файла, недопустимое значение. Механизм обработки исключений позволяет перехватить ошибку и завершить сценарий предсказуемо, не прерывая работу программы аварийно.

- конструкция `try` содержит код, в котором может возникнуть ошибка;
- блок `except` перехватывает и обрабатывает конкретный тип исключения;
- инструкции `else` и `finally` дополняют обработку: первая выполняется при отсутствии ошибок, вторая — в любом случае;
- типичные исключения: `ValueError`, `TypeError`, `KeyError`, `FileNotFoundError`.

В индивидуальном проекте исключения обрабатывают некорректный ввод пользователя и ошибки при работе с файлами данных — это обязательное требование к программной части ПР2.

4.6. Работа с файлами и контекстные менеджеры

Файлы позволяют сохранять данные между запусками программы. В учебном проекте удобно использовать формат JSON: он поддерживается стандартным модулем json и хорошо подходит для хранения списков и словарей. Чтение и запись файла выполняются через контекстный менеджер with, который автоматически закрывает файл после завершения работы.

- открытие файла выполняется функцией `open()` с указанием режима: чтение или запись;
- конструкция `with` гарантирует закрытие файла даже при возникновении ошибки;
- модуль `json` преобразует словари и списки Python в текстовый формат и обратно;
- данные приложения хранятся в отдельном каталоге, например `data/`.

Применение файлов в индивидуальном проекте делает поведение приложения естественным: данные, добавленные пользователем, сохраняются и доступны при следующем запуске.

4.7. Расширенные возможности Python

Тема «Расширенные возможности Python» объединяет механизмы, которые применяются по мере усложнения программ. На ПР2 достаточно уверенного знакомства с ними и корректного применения в учебных заданиях и индивидуальном проекте:

- области видимости и пространства имён определяют, где доступны переменные; локальные переменные функции изолированы от остального кода;
- итераторы и генераторы обеспечивают последовательный перебор данных; генератор формирует значения по мере необходимости;
- lambda-функции — короткие безымянные функции; применяются, например, для задания ключа сортировки;
- интроспекция — исследование объектов во время выполнения: просмотр атрибутов, функций и документации помогает понимать и отлаживать код;
- документирование кода (docstring, комментарии, `README.md`) делает проект понятным для других разработчиков.

Перечисленные возможности не требуют искусственного применения везде, где это возможно. Используйте их там, где они делают решение понятнее: генератор — для отбора подходящих элементов, lambda-функцию — для сортировки, интроспекцию — при отладке.

Ограничение. Декораторы в рамках ПР2 не изучаются: они рассматриваются на ПР3 совместно с объектно-ориентированным программированием.

5. ПРАКТИЧЕСКАЯ ЧАСТЬ. ВЫПОЛНЕНИЕ МАТЕРИАЛОВ ЯНДЕКС ПРАКТИКУМА

Практическая часть начинается с учебных заданий Яндекс Практикума: сначала повторяются и закрепляются основные конструкции языка, затем выполняется проект «Холодильник», после этого изученные возможности применяются при доработке индивидуального проекта. Подразделы 5.1–5.3 выполняются до перехода к разделу 6. Подразделы 5.4–5.6 описывают завершающие шаги практической части — тестирование и отладку программы, обновление файла README.md и фиксацию результата

в Git — и выполняются после доработки индивидуального проекта по материалам раздела 6.

5.1. Повторение и закрепление основных конструкций Python

Выполните в Яндекс Практикуме упражнения по темам раздела 01 «Основы Python» в полном объеме:

- коллекции;
- итерации и циклы;
- функции.

Особое внимание уделите использованию:

- списков, кортежей, множеств и словарей;
- циклов `for` и `while`;
- функций с параметрами и возвращаемыми значениями;
- обработки и преобразования данных с использованием функций и циклов.

Упражнения Практикума снабжены автоматической проверкой: если решение не принимается, проанализируйте сообщение проверки, при необходимости воспользуйтесь отладчиком IDE и материалами раздела 4 настоящих указаний.

5.2. Проект «Холодильник»

Выполните проектную работу «Холодильник» в Яндекс Практикуме. Программа ведет учет продуктов и предоставляет пользователю информацию о содержимом холодильника.

Программа должна:

- хранить перечень продуктов;
- хранить количество продуктов;
- учитывать срок годности;
- позволять добавлять продукты;
- предоставлять информацию о содержимом холодильника.

При выполнении проекта должны использоваться:

- словари;
- целые и дробные числа;
- функции;
- циклы.

Результатом выполнения является работоспособная программа, соответствующая требованиям задания Практикума. Обратите внимание, как именно словари применяются для хранения данных: тот же прием понадобится в индивидуальном проекте.

5.3. Расширение возможностей программы

После выполнения основной части проекта примените возможности Python, изученные в уроках темы «Python: новый уровень», для организации и доработки программы. Выполните в соответствии с материалами Практикума задания, связанные с:

- разбиением программы на модули;
- организацией модулей и пакетов;
- обработкой исключительных ситуаций;
- чтением и записью файлов;
- использованием контекстных менеджеров;
- документированием программного кода;
- использованием средств интроспекции.

Дополнительные возможности языка, изучаемые в рамках ПР2:

- области видимости и пространства имён;
- итераторы и генераторы;
- lambda-функции.

Последовательность «решение в одном файле — разделение на модули — обработка ошибок — сохранение данных» повторяет реальный путь развития приложения. Пройдите ее на проекте «Холодильник», чтобы осознанно применить те же приемы в собственном проекте.

5.4. Тестирование и отладка программы

Запустите программу индивидуального проекта (сквозной пример — «Система бронирования помещений», раздел 6) и проверьте работу всех пунктов меню на подготовленных данных. Пример меню сквозного примера:

```
=== Система бронирования помещений ===
1. Показать помещения
2. Найти помещение по названию
3. Проверить вместимость
4. Проверить доступность на дату
5. Забронировать помещение
6. Отменить бронирование
7. Показать бронирования
0. Выход
```

В ходе тестирования необходимо:

- проверить каждую функцию меню на корректных данных;
- убедиться, что ввод некорректных значений (буква вместо числа, неверная дата, несуществующий номер) не приводит к аварийному завершению;
- проверить сохранение данных: после добавления и отмены бронирования файлы JSON должны обновляться;

- исследовать программу в отладчике: установить точку останова внутри функции проверки доступности и проследить значения переменных при переборе списка.

5.5. Обновление файла README.md

Дополните файл README.md индивидуального проекта: опишите реализованные на ПР2 функции, структуру модулей и формат данных. Актуализируйте разделы «Возможности» и «Структура проекта», добавьте команды запуска программы, запуска тестов и проверки качества кода.

Итоговый вариант README.md сквозного примера приведен в подразделе 6.2 — используйте его как образец при обновлении собственного файла. README.md является частью индивидуального проекта и будет дополняться в последующих практических работах.

5.6. Фиксация результата в Git

После завершения работы зафиксируйте результат в репозитории:

- проверьте состояние рабочего дерева;
- добавьте необходимые файлы (включая модули, файлы данных и тесты);
- создайте commit с информативным сообщением;
- проверьте историю изменений;
- отправьте изменения в удаленный репозиторий, если он используется.

Результат ПР2 должен быть полностью сохранен в репозитории: это позволяет преподавателю проверить ход развития проекта и готовит основу для следующего этапа разработки.

6. ИНДИВИДУАЛЬНЫЙ СКВОЗНОЙ ПРОЕКТ

В рамках ПР2 продолжается разработка индивидуального проекта, начатого в ПР1. Существующий проект не создается заново: начальный программный сценарий ПР1 перерабатывается и расширяется за счет коллекций, функций, циклов, обработки исключений и хранения данных в файлах.

Подраздел 6.1 определяет обязательные требования к доработке проекта, подраздел 6.2 последовательно показывает сквозной пример доработки — от развития кода из ПР1 до первичного тестирования и итогового файла README.md, подраздел 6.3 приводит ожидаемый результат для самопроверки.

6.1. Обязательная доработка индивидуального проекта

В существующем репозитории индивидуального проекта реализуйте программную часть, соответствующую изученным на ПР2 темам. В проекте должны быть представлены:

- структуры данных для хранения информации предметной области;
- функции обработки данных;

- использование коллекций;
- циклы для обработки наборов данных;
- разбиение программного кода на несколько модулей;
- обработка предусмотренных программой исключительных ситуаций;
- сохранение и загрузка данных проекта в JSON-файлах.

Реализация должна демонстрировать практическое применение изученных в Яндекс Практикуме возможностей Python: проверяющий должен находить в коде проекта те конструкции, которые изучались в рамках ПР2, и понимать, какую задачу каждая из них решает. Начальный сценарий ПР1 при этом не удаляется, а перерабатывается — подраздел 6.2 показывает, как это делается на сквозном примере.

Требования к коду. При выполнении ПР2 примените к индивидуальному проекту требования к качеству программного кода, изученные в соответствующем разделе курса. Код доработанного проекта должен быть аккуратно оформлен, проверен линтером и снабжен аннотациями типов основных функций.

1. Оформление кода. Код проекта должен соответствовать следующим требованиям:

- понятная структура программы: код разделен на модули по смыслу;
- осмысленные имена переменных и функций: имя отражает назначение;
- логические модули: каждый модуль отвечает за свою часть функциональности;
- соответствие стандартам оформления Python (PEP 8);
- отсутствие неиспользуемого кода: закомментированных фрагментов, неиспользуемых переменных и импортов;
- отсутствие дублирующихся функций: повторяющаяся логика выделяется в отдельную функцию.

2. Проверка линтером. Код проекта проверяется линтером flake8. Линтер выявляет отклонения от стандарта оформления, неиспользуемые импорты и переменные, превышение допустимой длины строки и другие замечания. Проверка выполняется из терминала в каталоге проекта:

```
cd room_booking
flake8
```

Если замечаний нет, flake8 не выводит ничего — это ожидаемый результат. При наличии замечаний исправьте их: в сообщении указываются файл, номер строки и тип замечания, после чего проверка выполняется повторно.

Примечание. Отдельный скриншот результата проверки линтером в отчет не включается: соблюдение требований подтверждается состоянием кода в репозитории и может быть продемонстрировано на защите.

3. Аннотации типов. Основные функции проекта снабжаются аннотациями типов — указаниями типов параметров и возвращаемого значения непосредственно в сигнатуре функции. Аннотации делают соглашения о данных явными и помогают обнаруживать ошибки. Пример оформления сигнатур функций сквозного примера:

```
def add_room(
    rooms: dict[int, dict],
    room_name: str,
    capacity: int
) -> None:
    ...
def create_booking(
    bookings: list[dict],
    room_id: int,
    booking_date: date
) -> dict:
    ...
```

Ограничение. Полное строгое типизирование всего кода на данном этапе не требуется: аннотации добавляются к основным функциям проекта, где они описывают передаваемые и возвращаемые данные.

6.2. Пример доработки: проект «Система бронирования помещений»

В качестве сквозного примера, как и в ПР1, рассмотрим индивидуальный проект «Система бронирования помещений». В ПР1 в проекте была реализована проверка возможности бронирования одного помещения на определенную дату на основе отдельных переменных. В ПР2 этот вариант дорабатывается с учетом изученных возможностей Python: данные переводятся в структуры-коллекции, функциональность выделяется в отдельные функции и модули, добавляются обработка ошибок и работа с файлами.

Важно. Приведенный пример показывает возможный вариант организации проекта. Он не является готовым решением и не требует полного копирования. Состав функций и файлов может изменяться в зависимости от предметной области индивидуального проекта.

Развитие кода из ПР1

В ПР1 был реализован следующий первоначальный сценарий:

```
from datetime import date


room_name = "Аудитория 301"
capacity = 30
booking_date = date(2026, 9, 15)
is_available = True
```

```
def get_booking_status(is_available):
    if is_available:
        return "Помещение доступно для бронирования"

    return "Помещение уже занято"



print(f"Помещение: {room_name}")
print(f"Вместимость: {capacity} человек")
print(f"Дата: {booking_date}")
print(get_booking_status(is_available))
```

После ПР2 этот код не должен просто исчезнуть — он должен быть переработан. Первоначальные одиночные переменные `room_name`, `capacity`, `booking_date`, `is_available` превращаются в структуры данных проекта: участки данных приложения переносятся из переменных программы в коллекции. Например:

```
rooms      # помещения проекта
bookings   # бронирования проекта
```

Здесь `rooms` — словарь помещений: ключ — идентификатор помещения, значение — данные помещения (название, вместимость); `bookings` — список бронирований, каждое из которых представлено словарем.

Функция get_booking_status() остается частью проекта, но теперь используется вместе с другими функциями:

```
add_room()
find_room()
check_room_capacity()
is_room_available()
create_booking()
get_booking_status()
```

Например, вывод статуса помещения теперь выполняется по результату проверки доступности — функция из ПР1 вызывается вместе с новой функцией:

```
print(get_booking_status(
    is_room_available(bookings, room_id, booking_date)
))
```

Преемственность этапов развития проекта:

- ПР1 — проверка одного сценария бронирования;

- ПР2 — создание функциональной программы, работающей с множеством помещений и бронирований;
- ПР3 — преобразование предметной области в систему объектов и классов.

Структура проекта

Программа разделяется на несколько модулей, данные приложения выносятся в отдельный каталог, тесты — в отдельный каталог tests:

```
room_booking/
│
├── README.md
├── requirements.txt
├── main.py
├── rooms.py
├── bookings.py
├── storage.py
├── utils.py
├── data/
│   ├── rooms.json
│   └── bookings.json
│
└── tests/
    ├── test_rooms.py
    └── test_bookings.py
```

Назначение файлов проекта:

Файл	Назначение
main.py	Точка запуска программы и основной сценарий взаимодействия
rooms.py	Функции для работы с помещениями
bookings.py	Функции для создания и проверки бронирований
storage.py	Функции сохранения и загрузки данных
utils.py	Вспомогательные функции
data/rooms.json	Данные о помещениях
data/bookings.json	Данные о бронированиях
tests/test_rooms.py	Автоматические тесты функций работы с помещениями
tests/test_bookings.py	Автоматические тесты функций бронирования
README.md	Документация проекта
requirements.txt	Список зависимостей проекта

Основной код проекта использует только стандартную библиотеку Python, поэтому программных зависимостей у проекта нет. Файл requirements.txt фиксирует инструменты разработки — pytest и flake8, применяемые при тестировании и проверке качества кода:

```
pytest>=8.0
flake8>=7.0
```

Установка зависимостей выполняется одной командой: `pip install -r requirements.txt`. Такой способ фиксирует точный состав окружения проекта: проект можно развернуть на другом компьютере, не изучая документацию.

Приведенная структура является рекомендуемой и может адаптироваться под выбранную тему проекта: названия модулей должны отражать предметную область. Разделение кода на модули применяет материал темы «Модули и пакеты», каталог tests заполняется при первичном тестировании проекта, которое рассматривается ниже в этом подразделе.

Функции проекта

Функции сквозного примера и их распределение по модулям приведены в таблице.

Модуль	Функция	Назначение
main.py	main()	Точка запуска: меню приложения и вызов функций проекта
main.py	show_rooms()	Вывод списка помещений
main.py	show_bookings()	Вывод списка бронирований
rooms.py	add_room()	Добавление помещения в словарь rooms
rooms.py	find_room()	Поиск помещений по подстроке названия
rooms.py	check_room_capacity()	Проверка вместимости помещения
rooms.py	filter_rooms_by_capacity()	Отбор помещений по вместимости
rooms.py	sort_rooms()	Сортировка помещений
bookings.py	is_room_available()	Проверка доступности помещения на дату
bookings.py	create_booking()	Создание бронирования
bookings.py	cancel_booking()	Отмена бронирования
bookings.py	get_booking_status()	Текстовый статус помещения (функция из ПР1)
storage.py	load_rooms() / save_rooms()	Загрузка и сохранение помещений (JSON)
storage.py	load_bookings() / save_bookings()	Загрузка и сохранение бронирований (JSON)
utils.py	input_int() / input_date()	Ввод чисел и дат с обработкой ошибок

Состав функций может отличаться от приведенного: в вашем проекте имена функций должны отражать предметную область. Стремитесь к тому, чтобы каждая функция выполняла одну логически завершенную операцию. Отдельные возможности языка применяются внутри функций проекта там, где это уместно: например, `sort_rooms()` может использовать lambda-функцию в качестве ключа сортировки, а

`filter_rooms_by_capacity()` — генератор для отбора подходящих помещений. Сигнатуры основных функций снабжаются аннотациями типов (подраздел 6.1).

Заготовки функций проекта

Для каждой функции, приведенной в таблице, ниже указана заготовка: сигнатура, документирующая строка и комментарий TODO с описанием того, что нужно реализовать. На данном этапе приводится структура функций, но не их готовая реализация: тела функций реализуются самостоятельно в ходе выполнения работы.

Модуль rooms.py объединяет функции работы с помещениями:

```
# rooms.py
def add_room(rooms, room_name, capacity):
    """Добавить помещение в словарь rooms.

    TODO: сформировать идентификатор помещения
    и добавить его данные в словарь rooms.
    """
    pass



def find_room(rooms, query):
    """Найти помещения по подстроке названия.

    TODO: перебрать словарь rooms и отобрать помещения,
    в названии которых встречается подстрока query.
    """
    pass



def check_room_capacity(rooms, room_id, min_capacity):
    """Проверить вместимость помещения.

    TODO: вернуть True, если вместимость помещения
    не меньше значения min_capacity.
    """
    pass



def filter_rooms_by_capacity(rooms, min_capacity):
    """Отобрать помещения по вместимости.

    TODO: перебрать словарь rooms и вернуть помещения,
    вмещающие не меньше min_capacity человек.
```

```
    """
    pass


def sort_rooms(rooms):
    """Отсортировать помещения по вместимости.

    TODO: вернуть помещения, упорядоченные по вместимости
    (например, с использованием lambda-функции).
    """
    pass
```

Модуль bookings.py содержит функции работы с бронированиями; функция get_booking_status() переносится из ПР1 без изменений:

```
# bookings.py
def is_room_available(bookings, room_id, booking_date):
    """Проверить, свободно ли помещение на дату.

    TODO: перебрать список bookings и вернуть False,
    если бронирование помещения на эту дату уже есть.
    """
    pass


def create_booking(bookings, room_id, booking_date):
    """Создать новое бронирование.

    TODO: проверить доступность помещения и добавить
    словарь бронирования в список bookings.
    """
    pass


def cancel_booking(bookings, booking_id):
    """Отменить бронирование.

    TODO: найти бронирование по идентификатору
    и удалить его из списка bookings.
    """
    pass


def get_booking_status(is_available):
```

```
    """Вернуть текстовый статус помещения."""
    if is_available:
        return "Помещение доступно для бронирования"

    return "Помещение уже занято"
```

Модуль storage.py отвечает за сохранение и загрузку данных: чтение и запись JSON-файлов выполняются через контекстный менеджер with, ошибки файлов обрабатываются исключениями:

```
# storage.py
def load_rooms(filename):
    """Загрузить помещения из JSON-файла.

    TODO: открыть файл через контекстный менеджер with,
    прочитать данные и вернуть список помещений.
    Обработать отсутствие файла и некорректный JSON.
    """
    pass


def save_rooms(filename, rooms):
    """Сохранить помещения в JSON-файл.

    TODO: записать список помещений в файл
    с использованием контекстного менеджера with.
    """
    pass


def load_bookings(filename):
    """Загрузить бронирования из JSON-файла.

    TODO: аналогично load_rooms() — прочитать
    и вернуть список бронирований.
    """
    pass


def save_bookings(filename, bookings):
    """Сохранить бронирования в JSON-файл.

    TODO: записать список бронирований в файл.
    """
```

```
    pass
```

Модуль utils.py содержит функции безопасного ввода: при некорректном вводе запрос повторяется, программа не завершается аварийно:

```
# utils.py
def input_int(prompt):
    """Запросить у пользователя целое число.

    TODO: прочитать ввод и преобразовать его в число;
    при некорректном вводе повторить запрос.
    """
    pass



def input_date(prompt):
    """Запросить у пользователя дату.

    TODO: прочитать строку и преобразовать в объект date;
    при некорректном формате повторить запрос.
    """
    pass
```

Модуль main.py содержит функции вывода данных и точку запуска приложения:

```
# main.py
def show_rooms(rooms):
    """Вывести список помещений.

    TODO: перебрать словарь rooms и вывести помещения
    в виде таблицы.
    """
    pass



def show_bookings(bookings):
    """Вывести список бронирований.

    TODO: перебрать список bookings и вывести бронирования
    с указанием помещения и даты.
    """
    pass



def main():
```

```
    """Точка запуска приложения.

    TODO: реализовать цикл меню и вызов функций проекта
    в зависимости от выбора пользователя.
    """
    pass


if __name__ == "__main__":
    main()
```

Конструкция `if __name__ == "__main__"` запускает меню только при непосредственном запуске файла main.py: при импорте модуля, например из тестов, меню не выполняется (тема «Модули и пакеты»). Заготовки задают только структуру: при реализации снабдите основные функции аннотациями типов (подраздел 6.1), сохраните документирующие строки и добейтесь прохождения первичных тестов, которые рассматриваются ниже в этом подразделе.

Пример данных rooms.json

Данные о помещениях хранятся в файле data/rooms.json в виде списка словарей:

```
[
    {
        "id": 1,
        "name": "Аудитория 301",
        "capacity": 30
    },
    {
        "id": 2,
        "name": "Конференц-зал",
        "capacity": 60
    }
]
```

Данные о бронированиях хранятся аналогично в файле data/bookings.json:

```
[
    {
        "id": 1,
        "room_id": 1,
        "booking_date": "2026-09-15"
    }
]
```

Примечание. Запись бронирования содержит только идентификатор, помещение и дату — данных о клиенте в ней нет. Это осознанное упрощение: все проверки ПР2 — доступность помещения и запрет повторного бронирования — зависят только

от помещения и даты, поэтому данные о клиенте не участвуют в логике этого этапа. Клиент как участник предметной области требует собственного описания: перечня клиентов, связи «бронирование — клиент», поиска бронирований по клиенту. Эти вопросы рассматриваются на ПР3, когда предметная область преобразуется в систему объектов и классов.

При запуске программа читает файлы, при изменениях — сохраняет их заново. Чтение и запись выполняются в модуле storage.py с использованием контекстного менеджера with и обработкой исключений (отсутствие файла, некорректный JSON).

Пример работы приложения

К окончанию ПР2 приложение может предоставлять, например, следующий сценарий:

```
=== Система бронирования помещений ===


1. Показать помещения
2. Найти помещение по названию
3. Проверить вместимость
4. Проверить доступность на дату
5. Забронировать помещение
6. Отменить бронирование
7. Показать бронирования
0. Выход

Выберите действие: 4

Название помещения: Аудитория 301
Дата: 15.09.2026

Помещение уже занято
```

При проверке другой даты:

```
Название помещения: Аудитория 301
Дата: 20.09.2026

Помещение доступно для бронирования
```

Статус помещения выводится функцией get_booking_status(), перенесенной из ПР1, по результату проверки is_room_available().

Первичное тестирование проекта

Настройте первичное автоматизированное тестирование проекта с использованием pytest. Автоматизированные тесты проверяют работу функций без ручного

запуска программы и подтверждают корректность реализации после доработки проекта.

Порядок настройки:

- установите pytest в рабочее окружение: `pip install pytest` (или установите зависимости проекта из файла requirements.txt);
- создайте в проекте каталог `tests` (предусмотрен структурой проекта);
- создайте не менее одного файла тестов, например `tests/test_rooms.py`;
- реализуйте 3–5 простых автоматизированных тестов;
- запустите тесты из корня проекта командой `pytest`.

В первую очередь тестируйте функции, которые не требуют сложной настройки окружения: функции обработки данных, принимающие коллекции и возвращающие результат. Каждый тест проверяет одну функцию на одном сценарии и завершается проверкой assert.

Примеры тестов функций помещений — добавление, поиск, проверка вместимости:

```
# tests/test_rooms.py
from rooms import add_room, find_room, check_room_capacity



def test_add_room():
    rooms = {}
    add_room(rooms, "Аудитория 301", 30)
    assert len(rooms) == 1



def test_find_room():
    rooms = {}
    add_room(rooms, "Аудитория 301", 30)
    assert find_room(rooms, "аудитория")



def test_check_room_capacity():
    rooms = {}
    add_room(rooms, "Конференц-зал", 60)
    assert check_room_capacity(rooms, 1, 50)
```

Тесты бронирования — проверка доступности помещения и запрет повторного бронирования на ту же дату:

```
# tests/test_bookings.py
from datetime import date
```

```
from bookings import is_room_available, create_booking


def test_is_room_available():
    bookings = []
    assert is_room_available(bookings, 1, date(2026, 9, 15))


def test_duplicate_booking_forbidden():
    bookings = []
    create_booking(bookings, 1, date(2026, 9, 15))
    assert not is_room_available(bookings, 1, date(2026, 9, 15))
```

Результат выполнения тестов выводится в терминал: все тесты должны завершиться со статусом passed. Имена функций в примерах соответствуют сквозному примеру; в своем проекте используйте имена собственных функций, сохраняя общий принцип: один тест — одна функция — один сценарий.

Каталог tests с файлами тестов является частью репозитория и фиксируется в Git вместе с остальным кодом проекта.

Применение материала курса

Соответствие материала курса и его реализации в проекте приведено в таблице. Названия материалов указаны в терминах Яндекс Практикума: материалу «ООП и другие полезные вещи» соответствует раздел 03 «Python: новый уровень», материалам «Требования к коду» и «Pytest» — требования к коду (подраздел 6.1) и первичное тестирование проекта.

Материал курса	Что должно быть показано в проекте
«Основы Python»	коллекции, функции, итерации, циклы
«ООП и другие полезные вещи»	модули и пакеты, интроспекция, документирование, исключения, файлы, контекстные менеджеры
«Расширенные возможности»	пространство имён, итераторы и генераторы, lambda-функции (при целесообразности)
«Требования к коду»	оформление кода, проверка линтером, аннотации типов
«Pytest»	настройка тестирования, первые автоматизированные тесты

Не требуется искусственно использовать каждую изученную конструкцию в пользовательской функциональности. Конкретный способ применения определяется задачами проекта.

Файл README.md

Файл README.md является обязательной частью репозитория проекта. К завершению ПР2 он должен описывать актуальное состояние проекта. Итоговый вариант README.md сквозного примера:

```
# Система бронирования помещений

Консольное приложение для учета помещений
и управления их бронированием.

## Возможности
- просмотр и поиск помещений;
- проверка вместимости;
- проверка доступности на дату;
- добавление бронирований;
- отмена бронирований.

## Структура проекта
- main.py — точка запуска;
- rooms.py — работа с помещениями;
- bookings.py — работа с бронированиями;
- storage.py — сохранение и загрузка данных;
- utils.py — вспомогательные функции;
- data/ — JSON-файлы данных;
- tests/ — автоматизированные тесты.

## Требования
- Python 3.x;
- pytest;
- flake8.

Установка зависимостей:
pip install -r requirements.txt

## Запуск программы
python main.py

## Запуск тестов
pytest

## Проверка качества кода
flake8
```

README.md актуализируется по мере развития проекта: в последующих практических работах в него добавляются описание новых возможностей и обновленной структуры проекта.

6.3. Ожидаемый результат

К окончанию ПР2 исходный проект из ПР1 должен быть доработан, а не заменен новым проектом: начальный сценарий сохраняется в составе приложения, а его данные и логика переводятся на новый уровень. В результате должна быть получена первая функциональная версия приложения, работающая с множеством помещений и бронирований.

Минимальный результат ПР2 — проверьте, что в проекте есть:

- репозиторий Git;
- структурированный проект;
- модули;
- функции обработки данных;
- коллекции;
- циклы;
- исключения;
- файлы;
- контекстный менеджер;
- документация функций;
- аннотации типов;
- код без замечаний flake8;
- каталог tests;
- тесты pytest;
- README.md.

При этом начальный сценарий ПР1 должен сохраниться в проекте: данные помещений и бронирований — в структурах rooms и bookings, функция get_booking_status() — в составе модуля bookings.py вместе с новыми функциями.

Важно. Классы и объекты на данном этапе не используются: данные представляются словарями и списками, поведение — функциями. Объектная модель будет введена при дальнейшем развитии проекта в ПР3, посвященной объектно-ориентированному программированию.

7. РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ ПРАКТИЧЕСКОЙ РАБОТЫ

К завершению ПР2 должны быть получены следующие результаты. Используйте этот перечень для самопроверки перед переходом к контролю: каждый пункт должен быть подтвержден демонстрацией или содержимым репозитория.

Материалы Яндекс Практикума

Раздел 01. Основы Python (продолжение):

- выполнена тема «Коллекции»;

- выполнена тема «Итерации и циклы»;
- выполнен проект «Холодильник».

Раздел 03. «Python: новый уровень»:

- выполнена тема «Модули и пакеты»;
- выполнена тема «Интроспекция»;
- выполнена тема «Документирование»;
- выполнена тема «Исключения»;
- выполнена тема «Файлы»;
- выполнена тема «Контекстные менеджеры»;
- изучены расширенные возможности языка.

Индивидуальный проект

- код разделен на модули;
- данные хранятся в JSON-файлах;
- реализованы функции обработки коллекций: добавление, поиск, отбор и сортировка помещений, вывод списков;
- реализованы проверка вместимости и доступности, создание и отмена бронирований;
- обработка исключений защищает программу от некорректного ввода и ошибок файлов;
- выполнено тестирование всех пунктов меню;
- выполнена отладка программы;
- код проверен линтером flake8, замечания устранены;
- основные функции снабжены аннотациями типов;
- созданы автоматизированные тесты pytest, все тесты проходят;
- обновлен README.md;
- результат сохранен в Git.

8. ФОРМА КОНТРОЛЯ И КРИТЕРИИ ОЦЕНКИ

Контроль выполнения ПР2 проводится индивидуально. Форма контроля включает:

- проверку выполнения материалов Яндекс Практикума;
- демонстрацию индивидуального проекта;
- проверку репозитория;
- проверку качества кода и первичных тестов;
- устный ответ на контрольные вопросы.

8.1. Чек-лист выполнения материалов Яндекс Практикума

Студент предъявляет преподавателю результат выполнения следующих материалов. Отмечайте пункты по мере выполнения – чек-лист используется и при самопроверке, и при контроле со стороны преподавателя.

Чек-лист раздела 01 «Основы Python» (продолжение)

Материал	Выполнено
Коллекции	☐
Итерации и циклы	☐
Проект «Холодильник»	☐

Чек-лист раздела 03 «Python: новый уровень»

Материал	Выполнено
Модули и пакеты	☐
Интроспекция	☐
Документирование	☐
Исключения	☐
Файлы	☐
Контекстные менеджеры	☐
Расширенные возможности	☐

Раздел 02 «Настройка рабочего окружения» выполнен на ПР1 и повторной проверке не подлежит. Таким образом, для допуска результата ПР2 к проверке должны быть отмечены все пункты приведенных чек-листов.

8.2. Чек-лист выполнения индивидуального проекта

Отмечайте пункты по мере выполнения — чек-лист используется при самопроверке и при контроле со стороны преподавателя.

Пункт проверки	Выполнено
код разделен на модули (rooms.py, bookings.py, storage.py, utils.py)	☐
данные хранятся в JSON-файлах и загружаются при запуске	☐
обработка исключений защищает программу от некорректного ввода	☐
оформление кода соответствует стандартам Python	☐
код проверен линтером flake8, замечания устранены	☐
основные функции снабжены аннотациями типов	☐
создан каталог tests с файлами тестов	☐
реализовано не менее 3–5 тестов pytest, все тесты проходят	☐
обновлен файл README.md	☐
результат зафиксирован в Git	☐

8.3. Критерии оценки

Работа считается выполненной, если по каждому показателю достигнут требуемый результат. Критерии используются преподавателем при защите результата работы.

№	Показатель	Что должно быть продемонстрировано
1	Выполнение материалов Яндекс Практикума	Отмеченные чек-листы разделов 01 и 03; ответы на вопросы по темам материалов
2	Применение коллекций	Использование списков и словарей для хранения и обработки данных проекта
3	Функциональная декомпозиция	Логика программы распределена по функциям и модулям; дублирование кода отсутствует
4	Обработка исключений	Некорректный ввод и ошибки файлов не приводят к аварийному завершению программы
5	Работа с файлами	Данные сохраняются в JSON-файлы и восстанавливаются при запуске
6	Развитие индивидуального проекта	Функциональность ПР2 интегрирована в существующий проект; структура и README.md актуальны
7	Фиксация результата	Репозиторий содержит рабочую версию проекта с историей изменений
8	Качество кода	Код оформлен по стандарту и проверен линтером flake8; основные функции имеют аннотации типов
9	Первичное тестирование	Установлен pytest; в каталоге tests реализовано не менее 3–5 тестов; тесты проходят

8.4. Требования к отчету

Отчет является документом, подтверждающим выполнение практической работы. Объем отчета – 1–2 страницы без учета титульной информации. В отчет включаются:

1. Общие сведения:

- дисциплина;
- номер и тема практической работы;
- группа;
- ФИО студента;
- год выполнения.

2. Выполненные материалы Яндекс Практикума

Раздел	Выполненные темы
Раздел 01. Основы Python (продолжение)	Коллекции; Итерации и циклы; проект «Холодильник»
Раздел 03. «Python:	Модули и пакеты; интроспекция; документирование;

новый уровень»	исключения; файлы; контекстные менеджеры; расширенные возможности

3. Индивидуальный проект. Указать:

- название проекта;
- перечень функций, реализованных на ПР2;
- структуру модулей проекта;
- формат хранения данных;
- ссылку на Git-репозиторий.

4. Результат работы. Кратко указать, что в результате выполнения ПР2:

- завершены темы раздела 01 и выполнен раздел 03;
- код проекта разделен на модули;
- данные переведены в JSON-файлы;
- реализованы новые функции и обработка исключений;
- выполнены тестирование и отладка.

5. Вывод. Сделать краткий вывод о результатах выполнения практической работы.

9. КОНТРОЛЬНЫЕ ВОПРОСЫ

При подготовке к защите используйте контрольные вопросы. Они сгруппированы по пяти направлениям и охватывают все разделы работы: материалы Яндекс Практикума, инструменты языка, качество кода, тестирование и индивидуальный проект. Ответ должен сопровождаться демонстрацией соответствующего участка кода проекта.

По коллекциям, циклам и функциям

1. Какие виды коллекций Python вы знаете? Для каких задач предназначен каждый из них?

2. Чем список отличается от кортежа?

3. В каких случаях удобно использовать множество?

4. Почему словари называются коллекцией пар «ключ – значение»?

5. Чем цикл for отличается от цикла while?

6. Для чего используется функция range()?

7. Как организовать перебор элементов словаря в цикле?

8. Для чего используются операторы break и continue?

9. Как объявить функцию в Python? Что возвращает функция без оператора return?

10. Чем позиционные параметры отличаются от именованных?

11. Что такое область видимости переменной?

12. Что такое docstring и для чего он нужен?

По модулям, исключениям и файлам

13. Что такое модуль Python?

14. Чем импорт модуля отличается от импорта отдельного объекта из модуля?

15. Что такое пакет и для чего нужен файл __init__.py?

16. Для чего используется конструкция try/except?

17. Чем блок finally отличается от блока else?

18. Для чего используется оператор raise?

19. Зачем закрывать файл после работы с ним?

20. Какие преимущества дает контекстный менеджер with?

По расширенным возможностям языка

21. Что такое интроспекция? Какие функции Python для нее используются?

22. Что такое пространство имен?

23. Что такое итератор и генератор?

24. Для чего используется оператор yield?

25. Что такое лямбда-функция и где она применяется?

По качеству кода и тестированию

26. Что такое аннотации типов и как они оформляются в сигнатуре функции?

27. Для чего используется линтер flake8 и как выполнить проверку кода проекта?

28. Какие требования к оформлению кода предъявляются в рамках ПР2?

29. Как устроен автоматизированный тест pytest и что проверяет оператор assert?

30. Как запустить тесты проекта и как интерпретировать результат их выполнения?

По индивидуальному проекту

31. Какие функции реализованы в проекте на этапе ПР2?

32. Как в проекте представлены данные (структуры, формат хранения)?

33. Как устроена проверка доступности помещения?

34. Какие исключения обрабатываются в программе и зачем?

35. Какие модули входят в состав проекта и как распределена логика между ними?

36. Какие конструкции Python применены в реализованных функциях?

37. Какие части проекта будут изменены после изучения ООП на ПР3?

38. Почему данные проекта хранятся в файлах, а не в переменных программы?

10. ИТОГ

В результате выполнения ПР2 завершено изучение базовых конструкций языка: освоены коллекции, циклы и функции, а также инструменты, необходимые при разработке крупных программ, – модули и пакеты, исключения, файлы и контекстные менеджеры. Полученные знания закреплены в проектном задании «Холодильник» и развитии индивидуального сквозного проекта: логика приложения оформлена в виде функций и распределена по модулям, данные переведены в JSON-файлы, реализованы обработка исключений, поиск помещений, проверка вместимости и доступности, создание и отмена бронирований. Код проекта приведен в соответствие с требованиями к качеству: оформление проверено линтером flake8, основные функции снабжены аннотациями типов, работа функций подтверждена первыми автоматизированными тестами pytest.

На следующей практической работе изучается объектно-ориентированное программирование: классы и объекты, декораторы и связанные возможности языка. В индивидуальном проекте объектная модель дополнит работу со словарями и функциями, поэтому к началу ПР3 важно, чтобы структура данных проекта была продумана, а функции – аккуратно оформлены и задокументированы.